

HelixAgent

HelixAgent

Не выбірайце адну мадэль — дазвольце ім дыскутаваць і адпраўляйце адказ, з якім яны згодныя.

Кароткае апісанне

HelixAgent — гэта гатовы да прадукцыйнага выкарыстання ансамблевы сэрвіс LLM на базе Go, які інтэлектуальна аб'ядноўвае адказы ад шматлікіх моўных мадэляў, уключаючы сістэму шматраундовых дэбатаў AI і дынамічны выбар пастаўшчыкоў на аснове праверкі, каб ствараць найбольш дакладныя і надзейныя вынікі.

Кароткае апісанне

HelixAgent — гэта ансамблевы сэрвіс LLM на базе Go, які аб'ядноўвае шматлікіх пастаўшчыкоў у адзін дакладны адказ. Ён праводзіць шматраундовыя дэбаты AI, дынамічна ацэньвае пастаўшчыкоў праз LLMsVerifier, маршрутызуе з дапамогай стратэгий, заснаваных на даверы, і падтрымлівае прадукцыйныя функцыі: кэшаванне, маніторынг, засцярогі бяспекі і API ў стылі OpenAI.

Падрабязнае апісанне

HelixAgent — гэта гатовы да прадукцыйнага выкарыстання ансамблевы сэрвіс LLM (MIT) на базе AI, які разглядае адказ адной мадэлі як гіпотэзу, а не як канчатковае рашэнне. Замест таго каб рабіць стаўку на аднаго пастаўшчыка, які можа памыліцца, быць схільным да зрушэнняў ці часова недаступным, ён аб'ядноўвае адказы ад некалькіх моўных мадэляў, каб дасягнуць найбольш дакладнага і надзейнага выніку. А калі пытанне настолькі складанае, што гэта апраўдана, ён запуская мадэль ў структураванай шматраундавай дыскусіі. Спіс удзельнікаў шырокі: у README

дакументаваны шматлікія пастаўшчыкі LLM у каталогу `internal/llm/providers/`, уключаючы Claude, DeepSeek, Gemini, Mistral, Qwen і xAI/Grok.

Ключавым з’яўляецца тое, што выбар пастаўшчыка не з’яўляецца статычным спісам пераваг — ён зарабляецца ў рэжыме рэальнага часу. Дынамічныя ацэнкі праверкі ад інтэграванага LLMsVerifier кіруюць маршрутызацыяй і забяспечваюць мяккае пераключэнне на найлепшага пастаўшчыка, а пры зніжэнні якасці аднаго з іх фіксуецца памылка з катэгарызацыяй. Аркестратар дэбатаў AI ператварае рознагалоссі ў сігнал: ён падтрымлівае некалькі тапалогій (сетка, зорка, ланцужок) і дысцыплінаваны пратакол фаз — Прапанова → Крытыка → Агляд → Сінтэз — з перакрываваным навучаннем паміж дэбатамі, каб сістэма з часам лепш умела ўзгадняць мадэлі. Стратэгіі маршрутызацыі ўключаюць выбар на аснове даверу, кансэнсус большасці і выяўленне семантычнай інтэнцыі, прычым усе адказы перадаюцца ў рэжыме рэальнага часу токенам за токенам, а не пасля таго, як увесць ансамбль сфармуецца.

Сэрвіс распрацаваны так, каб вытрымліваць нагрузкі прадукцыйнага асяроддзя, а не проста добра дэманстравацца: PostgreSQL і Redis утвараюць высокадаступны слой даных, Prometheus/Grafana/OpenTelemetry забяспечваюць метрыкі, панэлі кіравання і трасіроўку, а аўтэнтыфікацыя JWT, абмежаванне хуткасці, рухавік засцярог і выяўленне RII абгортваюць ансамбль у кантролі, неабходныя для рэальнага разгортвання. Ён арганізаваны прыкладна ў дваццаць асобных модуляў (EventBus, Observability, Auth, Storage, VectorDB, Embeddings, RAG, Memory, MCP і іншыя), кожны з якіх вырашае сваю задачу, і пастаўляецца з фрэймворкам аптымізацыі LLM (семантычнае кэшаванне, структураваная вывадка, палепшаная патокавая перадача) з інтэграцыямі для SGLang, LlamaIndex, LangChain, Guidance і LMQL. Паколькі канчатковыя кропкі і ансамблевая эндпойнты сумяшчальныя з OpenAI, існуючы кліент можа накіраваць запыт на HelixAgent і атрымаць ансамблевую развагі без перапісвання кода.

Змест

Чаму мы яго стварылі

Адзінкавы LLM можа быць памылковым, схільным да перадузятасці ці проста недаступным. HelixAgent быў створаны, каб дазволіць прыкладанням звяртацца да некалькіх мадэляў адначасова, зважаць на іх адказы згодна з вымеранай надзейнасцю і элегантна пераключацца на іншыя — ператвараючы ўразлівую залежнасць ад аднаго пастаўшчыка ў устойлівы, самаацэньвальны ансамбль.

Чаму гэта змяняе правілы гульні

Гэта аперацыяналізуе кансэнсус некалькіх мадэляў — пераносіць практыку “запытаць некалькі мадэляў і ўзгадніць іх адказы” з разрозненых скрыптоў у вытворчы сэрвіс. Замест таго, каб жорстка прاپісаць аднаго пастаўшчыка і спадзявацца на лепшае, каманды атрымліваюць маршрутызацыю на аснове жывых верагоднасных паказчыкаў, структураваны пратакол дэбатаў для пытанняў, дзе адна спроба недастатковая, і вытворчую ўстойлівасць (высокадаступны слой даных, поўную назіральнасць і засцерагальныя механізмы) — усё гэта за OpenAI-сумяшчальным API. Галоўны прарыў — прыняцце без парушэнняў: адна ўразлівая залежнасць ад пастаўшчыка ператвараецца ў устойлівы, самаацэньвальны ансамбль, а існуючыя кліенты пераходзяць на яго, проста змяніўшы канцавы пункт, а не код.

Што новага

- Структураваны шматраундовы AI дэбат, які разглядае нязгоду мадэляў як рэсурс: выбіраныя тапалогіі сеткі/зоркі/ланцужка, дысцыплінаваны пратакол Прапанова→Крытыка→Агляд→Сінтэз і навучанне на перасячэннях дэбатаў, якое назапашваецца з часам.
- Дынамічны выбар пастаўшчыка на аснове жывых LLMsVerifier паказчыкаў замест статычнага спісу пераваг — ансамбль накіроўвае запыты да таго, хто лепш працуе менавіта зараз, і элегантна пераключаецца на іншыя, калі адзін збочвае.
- Уласная Go платформа аптымізацыі LLM (семантычны кэш, структураваная вывадка, палепшаная трансляцыя), якая існуе самастойна, з магчымасцю падключэння знешніх аптымізатараў (SGLang, LlamaIndex, LangChain, Guidance, LMQL) па жаданні, а не па неабходнасці.
- Мадульная архітэктурна з прыкладна дваццацю адасобленымі модулямі, якая захоўвае адзеленасць задач і адкрывае шлях да вялікіх дадзеных, такіх як размеркаваная памяць і струменевая перадача графу ведаў.

Галоўныя тэхнічныя выклікі і як мы іх пераадолені

- **Выбар сярод многіх нераўнацэнных пастаўшчыкаў.** Пастаўшчыкі адрозніваюцца якасцю і змяняюцца з часам, таму любы фіксаваны рэйтынг ужо заўтра будзе памылковым. Мы вырашылі гэтую праблему, зрабіўшы выбар бесперапынна вымерным: LLMsVerifier паказчыкі

фарміруюць маршрутызацыю з улікам вагі даверу і большасці галасоў, з элегантным пераключэннем на іншыя, калі пастаўшчык пагаршае працу.

- **Атрыманне надзейнага адказу на сапраўды складаныя пытанні.** Адна мадэль, запытаная адзін раз, не мае механізму для выяўлення ўласнай памылкі. Аркестратар дэбатаў забяспечвае такі механізм — шматтапалагічны, фазавы дэбат (Прапанова → Крытыка → Агляд → Сінтэз), які вымушае мадэлі абвргаць і ўдасканалваць адна адну, перш чым будзе сфарміраваны канчатковы адказ.
- **Запуск ансамбля ў вытворчым асяроддзі, а не проста ў ноўтбуку.** Размнажэнне запытаў да многіх пастаўшчыкаў павялічвае паверхню адмоваў. Мы абмежавалі яе з дапамогай высокадаступнага слоя даных PostgreSQL+Redis, назіральнасці Prometheus/Grafana/OpenTelemetry для выпадкаў няправільнай працы пастаўшчыка ці маршруту, а таксама бяспечнага перыметра з аўтэнтыфікацыяй JWT, абмежаваннем хуткасці, рухавіком засцерагальных механізмаў і выяўленнем PII.

Змест

Тэхналагічны стэк

- **Go** — абраны таму, што паралельная адпраўка аднаго запыту некалькім пастаўшчыкам — гэта менавіта тое, для чаго прызначаны гаруціны, а разгортванне ў выглядзе адзінага бінарнага файла спрашчае дастаўку сэрвісу з ~20 мадулямі; ён ляжыць у аснове ўсяго сэрвісу і кожнага ўнутранага мадуля.
- **Gin (Web API)** — абраны дзеля хуткага і нізкаапаратнага HTTP-інтэрфейсу; абслугоўвае сумяшчальныя з OpenAPI канчатковыя кропкі /v1 для генерацыі, чату, стрымінгу і ансамбляў, што дазваляе існуючым кліентам выкарыстоўваць ансамбль без змен.
- **PostgreSQL** — абраны як надзейнае сховішча для сеансаў, аналітыкі і запісаў дэбатаў, каб кансэнсусныя рашэнні і гісторыя дыскусій былі падсправаздачнымі; ён забяспечвае высокую даступнасць даных.
- **Redis** — абраны дзеля кэшавання з нізкай затрымкай і чаргі задач; забяспечвае як кэшаванне адказаў, так і сямантычны слой кэша, які дазваляе паўтараемым ці амаль дубляваным запытам абыходзіць лішнія вылічэнні.
- **LLMsVerifier (інтэграваны)** — абраны, каб надзейнасць пастаўшчыкаў была вымернай велічынёй, а не здагадкай; яго паказчыкі ранжыруюць пастаўшчыкоў для маршрутызацыі і кіруюць пераключэннем на рэзервовы варыянт пры пагаршэнні працы.

- **Prometheus + Grafana + OpenTelemetry** — абраны, каб ансамбль, які ахоплівае многіх пастаўшчыкоў, заставаўся назіральным; яны падаюць метрыкі `helixagent_*`, панэлі маніторынгу і сквозную трасіроўку запытаў праз паралельную адпраўку.
- **Model Context Protocol (MCP) адаптары** — абраны дзеля пашыральнасці праз адкрыты пратакол; у README пералічаны шматлікія адаптары MCP для падключэння знешніх інструментаў і кантэксту.
- **Neo4j / ClickHouse / Kafka (BigData)** — абраны, каб пераадолець абмежаванні аднаго вузла: Neo4j і ClickHouse падтрымліваюць размеркаваную памяць і функцыі графу ведаў, а Kafka маштабуе стрымінг графа і даных падзей.
- **Інтэграцыі аптымізацыі (SGLang, LlamaIndex, LangChain, Guidance, LMQL)** — абраны, каб далучаць папярэдняе кэшаванне, пошук, дэкампазіцыю задач і абмежаваную генерацыю як дадатковыя сэрвісы, дзякуючы чаму больш складаныя аптымізацыі даступныя, але не абавязковыя.

Стан і заўвагі аб сумленнасці

- **Стан: бэта.** Сэрвіс апісваецца як гатовы да прадукцыйнага выкарыстання, але паказчыкі прадукцыйнасці і пакрыцця ў README (напрыклад, “1000+ запытаў/секунда”, “<500 мс з кэшам”, колькасць пастаўшчыкоў і скрыптоў валідацыі) з’яўляюцца самадэклараванымі паказчыкамі праекта, не правяранымі незалежна, і тут наўмысна пададзены ў якаснай форме.
- Колькасць пастаўшчыкоў у самім README вагаецца; на старонцы выкарыстоўваецца якасная фармулёўка “шмат пастаўшчыкаў”.

Прыярытэтны ўзровень: Helix-primary.